

Computer Networks

计网答辩PPT



第9组：廖子韬，胡耘宁，魏宇晴

目录

01. 设计介绍
Introduction to our design

02. 拥塞控制检查
Congestion Control check

03. 并发检查
Concurrency check & Crash check

04. 优化说明
Optimize check

设计介绍

Introduction to our design

01

添加的类

类名	类型	作用
PktType	IntEnum	定义协议包类型
DownloadSession	@dataclass	管理单个下载会话
UploadSession	@dataclass	管理单个上传会话
PeerState	普通类	统一管理所有全局状态
Peer	核心类	Peer 的主控制逻辑

数据结构（下载相关）

```
@dataclass 吕 不困了才有力气睡觉
class DownloadSession:
    """Represents a download session for a chunk from a specific peer."""
    chunk: str # current downloading chunk
    expected_seq: int = 1
    total_segs: Optional[int] = None
    start_time: float = field(default_factory=time.time)
    last_ack_time: float = field(default_factory=time.time)
    waiting_chunks: List[str] = field(default_factory=list)
    timeout_count: int = 0

    # RTT estimation for download timeout
    estimatedRTT: float = 0.5
    devRTT: float = 0.25
    timeoutInterval: float = 0.5
    last_packet_time: float = field(default_factory=time.time) # Track last packet arrival time
    last_probe_time: float = 0.0 # Track last probe ACK send time to avoid spam
```

```
# Downloads in progress: chunkhash -> byte buffer (collector)
self.receiving: Dict[str, bytes] = {}
```

```
# Out-of-order packet buffer
self.out_of_order_buffer: Dict[Tuple[Tuple[str, int], str], Dict[int, bytes]] = {}
```

每个 DownloadSession 对应一个“从某个 peer 下载特定 chunk”的独立会话，包含序号、超时与重试等运行时信息。

它保存了顺序接收、RTT 估计以及超时重试所需的全部状态。

用一个字典把 chunk hash 映射到已经接收到的字节流，用于数据重组和完整性校验。

乱序缓冲结构，用于保存提前到达的 DATA 包，直到 expected_seq 到达为止。

数据结构（上传相关）

```
@dataclass  # 不困了才有力气睡觉 *
class UploadSession:
    """Represents an upload session for a chunk to a specific peer."""
    addr: Tuple[str, int]
    chunk_bytes: bytes
    total_segs: int
    # RDT / timing
    last_sent_seq: int = 0
    last_sent_time: float = 0.0
    timeout: float = 0.5
    sent_times: Dict[int, float] = field(default_factory=dict)
    # ACK / seq window
    last_acked: int = 0
    send_base: int = 1
    next_seq_num: int = 1
    last_ack_time: float = field(default_factory=time.time) # Track when we last received an ACK
    # RTT estimation
    estimatedRTT: float = 0.5
    devRTT: float = 0.25
    timeoutInterval: float = 0.5
    # Congestion control (Tahoe-style)
    cc_state: str = "slow_start" # "slow_start" or "congestion_avoidance"
    ssthresh: int = 64
    cwnd: float = 1.0
    dupACKcount: int = 0
    fast_retransmitted: set = field(default_factory=set)
```

```
# Upload sessions: chunkhash -> session
self.uploads: Dict[Tuple[str, Tuple[str, int]], UploadSession] = {}
```

每个会话表示向某个 peer 发送一个 chunk 的应用层可靠传输器。它维护发送窗口、重传定时与 RTT 估计，并实现了拥塞控制以调节 cwnd 与 ssthresh。

uploads 字典让 peer 可以同时向多个不同 peer 上传不同的 chunk，每个上传会话完全隔离。

数据结构（chunk相关）

1. Chunk 表示方式:

chunk_hash: bytes / str → chunk_data: bytes

Chunk 在系统中以 hash 作为唯一标识，数据本身以二进制形式存储，便于网络传输与完整性校验。

2. 本地 Chunk 存储结构（PeerContext）

self.chunks : Dict[chunk_hash, bytes]

peer 启动时加载，用于回复 WHOHAS，用于处理 GET 请求并发送 DATA

3. 待下载 Chunk 集合（DOWNLOAD 触发）

want_chunks : List[chunk_hash]

由 process_download() 读取 chunk_file，决定后续 WHOHAS 广播内容

4. Chunk ↔ Peer 可用性映射

candidates : Dict[chunk_hash, List[peer_addr]]

该映射避免了重复广播 WHOHAS，提高 chunk 调度效率。

5. Chunk 下载状态

chunk → { requested | downloading | completed }

6. Chunk 数据重组缓冲（DATA / ACK 交互）

recv_buffer : Dict[seq, bytes]

DATA 包可能乱序到达，按 seq 重组完整 chunk。该结构保证 chunk 数据在不可靠网络下仍可正确重组。

报文头

```
HEADER_FMT: str = "BBHII"  
HEADER_LEN: int = struct.calcsize(HEADER_FMT)
```

所有数据包的头部都使用统一的、固定格式的二进制结构
Header 总长度: 12 bytes

顺序	字段	类型	字节数
1	Type Code	B	1 byte
2	Header Length	B	1 byte
3	Packet Length	H	2 byte
4	Sequence Number	I	4 byte
5	ACK Number	I	4 byte

TCP超时的判定、实现与行为

```
# 新增: 检查 WHOHAS 重试调度
if (self.state.next_whohas_retry_time is not None and
    now >= self.state.next_whohas_retry_time):
    self.state.next_whohas_retry_time = None
    self._retry_whohas()
```

```
pkt = _pack_header(PktType.WHOHAS, HEADER_LEN + len(payload), seq=0, ack=0) + payload

for p in self.context.peers:
    try:
        pid = int(p[0])
        if pid == self.context.identity:
            continue
        peer_ip = p[1]
        peer_port = int(p[2])
        self.sock.sendto(pkt, address=(peer_ip, peer_port))
    except Exception:
        continue
```

```
elapsed = now - last

# For concurrent sessions, we need to be more conservative:
# - If we've sent packets beyond send_base, ACK might just be delayed
# - Use a more conservative timeout (1.5x) in this case
if next_seq_num > send_base:
    # We've sent packets beyond send_base, ACK might be delayed
    # Use more conservative timeout to avoid false positives
    effective_timeout = timeout * 1.5
else:
    # We haven't sent beyond send_base, use normal timeout
    effective_timeout = timeout

if elapsed > effective_timeout:
```

1.WHOHAS重传
_retry_whohas()重传

2.上传超时
a. 是否发送了后续包-->延时/超时
b. _send_data_segment_for_session重传

TCP超时的判定、实现与行为

```
new_addr = cand_list[0]
try:
    payload = _hex_to_bytes(chunk_hex)
except ValueError:
    # 非法 hash, 放弃
    return

pkt_get = _pack_header(PktType.GET, HEADER_LEN + len(payload), seq=0, ack=0) + payload
self.sock.sendto(pkt_get, new_addr)
```

```
try:
    # Send WHOHAS for all remaining want_chunks
    payload = b"".join(_hex_to_bytes(h) for h in self.state.want_chunks)
    pkt = _pack_header(PktType.WHOHAS, HEADER_LEN + len(payload), seq=0, ack=0) + payload

    for p in self.context.peers:
        try:
            pid = int(p[0])
            if pid == self.context.identity:
                continue
            peer_ip = p[1]
            peer_port = int(p[2])
            self.sock.sendto(pkt, address=(peer_ip, peer_port))
        except Exception:
            continue
```

3. 下载超时

- 清空当前已接收chunk 并在候选列表里重新发送GET
- 重新对所有 want_chunks 发送 WHOHAS 等待新的 IHAVE

_check_timeouts()在run主函数里调用

其他与RDT实现相关的内容

ACK回复

允许发送方在收到ACK之前，在滑动窗口范围内连续发送多个数据包

```
effective_cwnd = int(cwnd)
# max_inflight = min(effective_cwnd, WINDOW_SIZE)
max_inflight = effective_cwnd
while (next_seq_num - send_base) < max_inflight and next_seq_num <= total_segs:
    self._send_data_segment_for_session(session_key, next_seq_num)
    session.next_seq_num = next_seq_num + 1
    next_seq_num += 1
```

滑动窗口

使发送方能处理损坏、重复的ACK，并让接收方能识别重复数据包

```
# Send ACK after processing in-order packet and all buffered packets
# This cumulative ACK tells sender the highest consecutive sequence number received
ack_pkt = _pack_header(PktType.ACK, HEADER_LEN, seq=0, ack=expected)
self.sock.sendto(ack_pkt, from_addr)
self._log(f"DEBUG: Sending ACK after buffered packets: {expected} to {from_addr}")
# Check if complete (after processing buffered packets)
current_expected = session.expected_seq
```

其他与RDT实现相关的内容

在_complete_download中确认数据完整性

```
data = self.state.receiving.get(chunk_hex)
if not data:
    self._log(f"DEBUG: No data in receiving for {chunk_hex}")
    return

# Check data size - allow >= CHUNK_DATA_SIZE (may have extra padding)
if len(data) < CHUNK_DATA_SIZE:
    self._log(
        f"DEBUG: Data size too small for {chunk_hex}: got {len(data)}, expected at least {CHUNK_DATA_SIZE}")
    return
```

```
sha1 = hashlib.sha1()
sha1.update(data)
computed_hash = sha1.hexdigest()

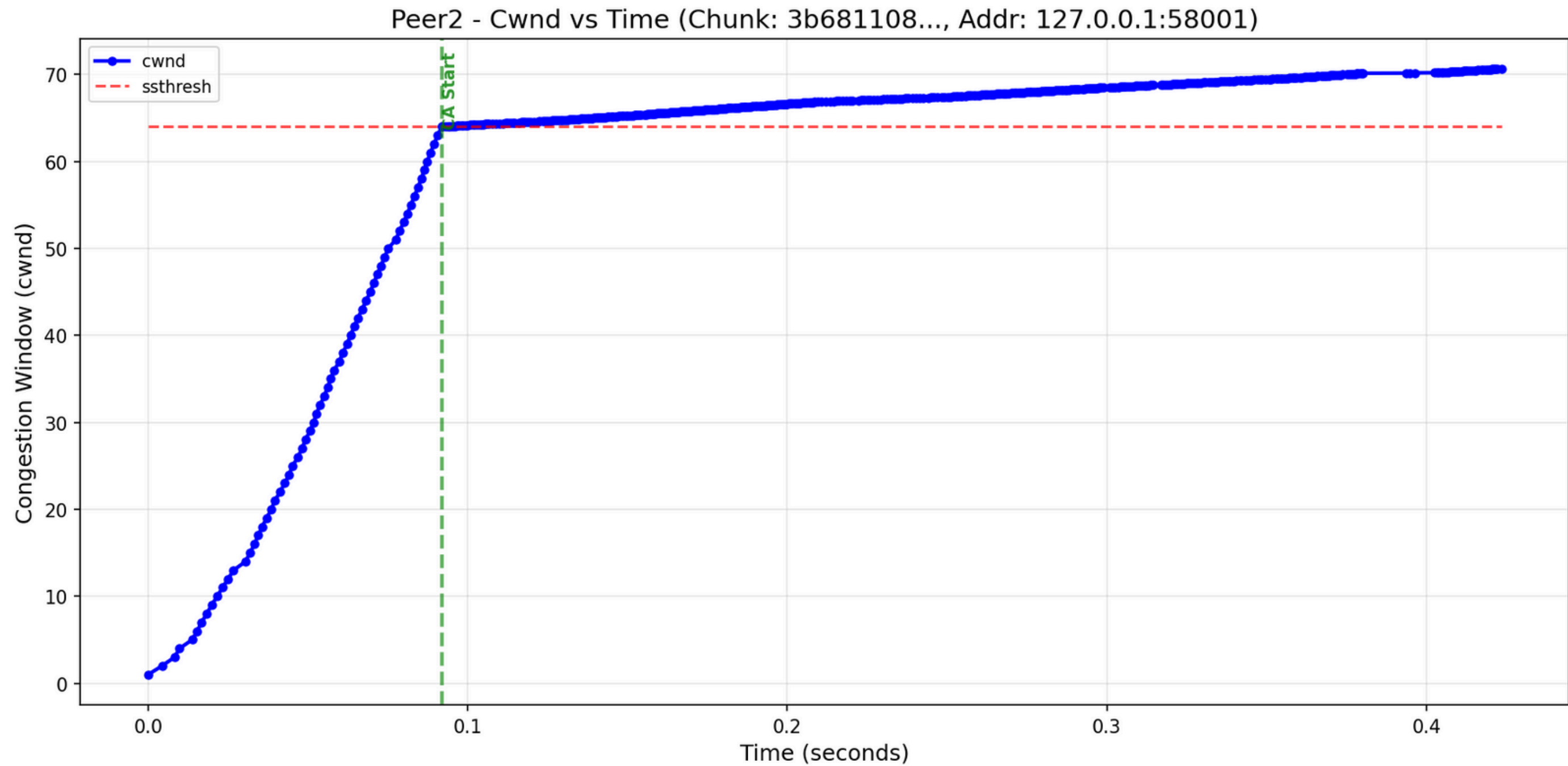
if computed_hash != chunk_hex:
    self._log(f"Hash mismatch! Expected {chunk_hex}, got {computed_hash}")
    return
```

拥塞控制检查

Congestion Control check

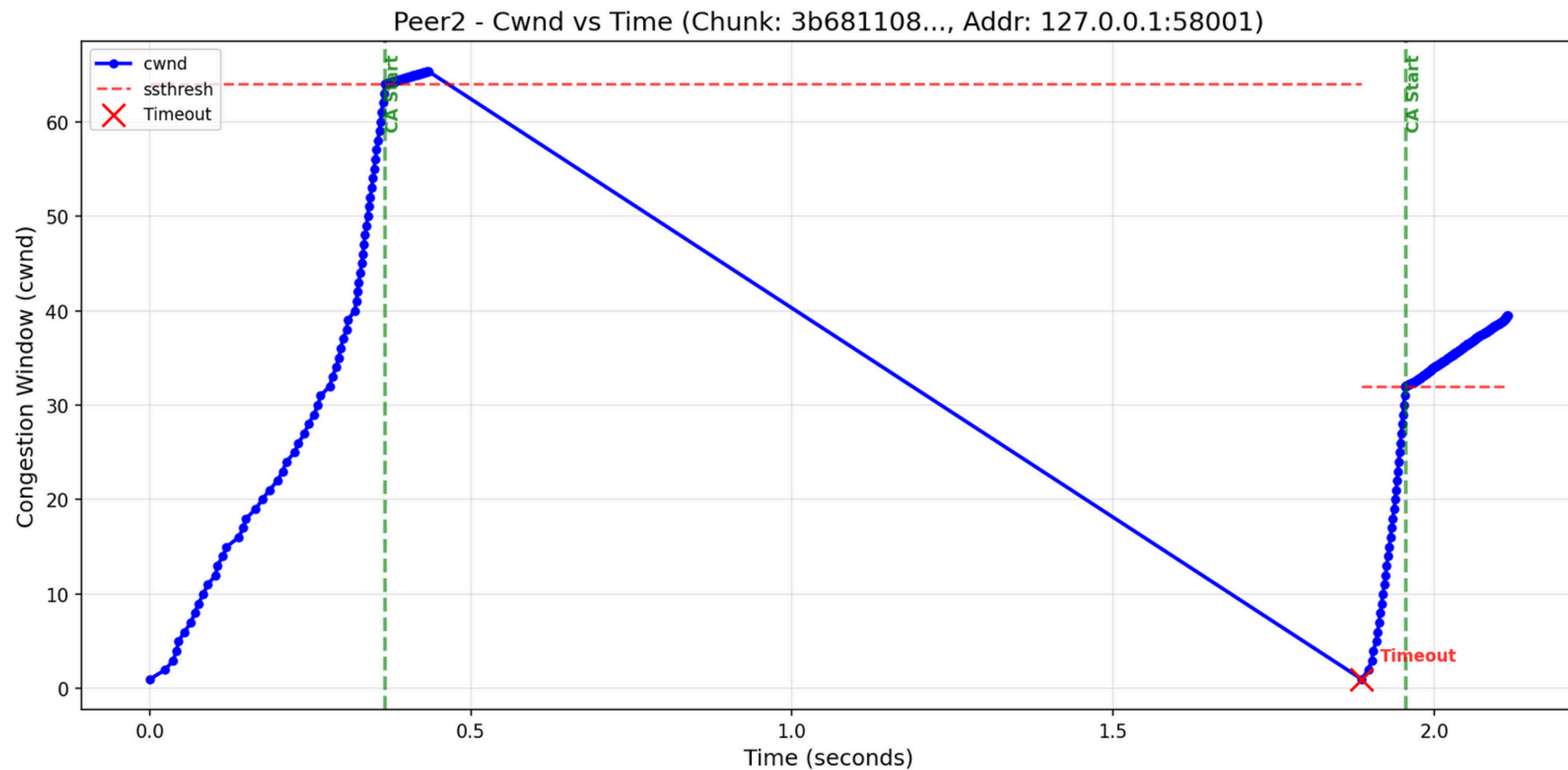
02

拥塞窗口记录图



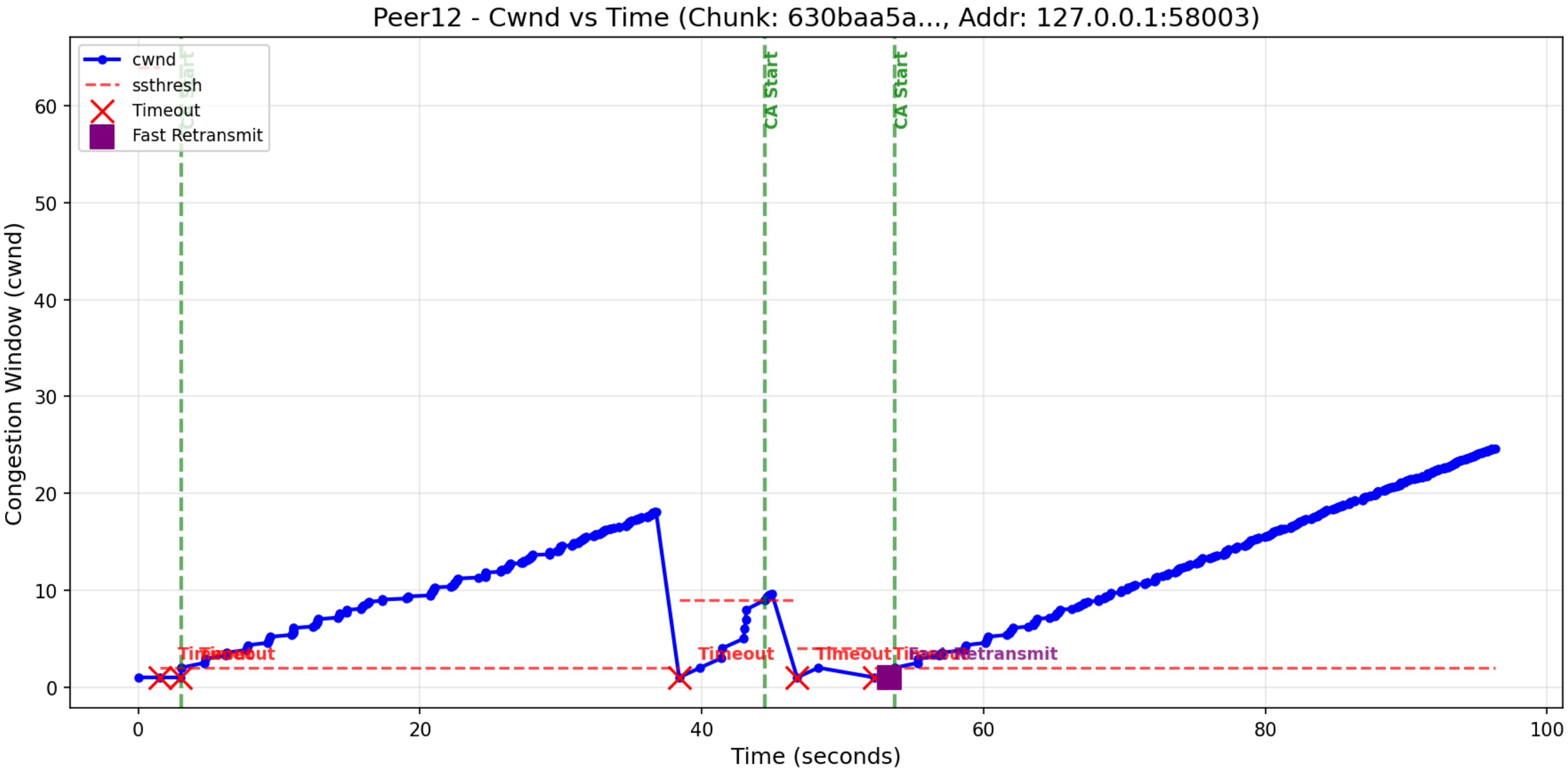
test01

拥塞窗口记录图



test02

拥塞窗口记录图



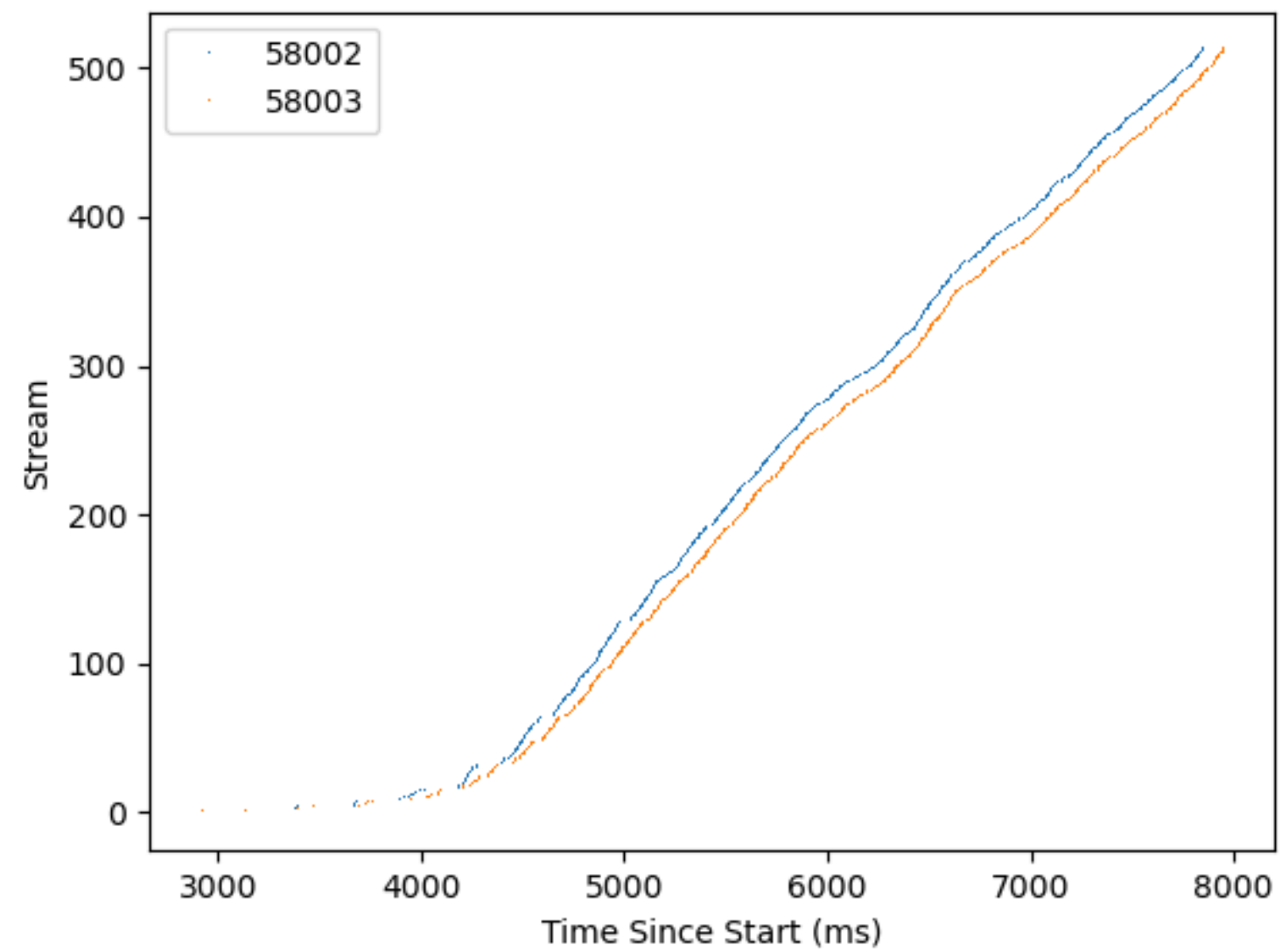
test06

并发检查

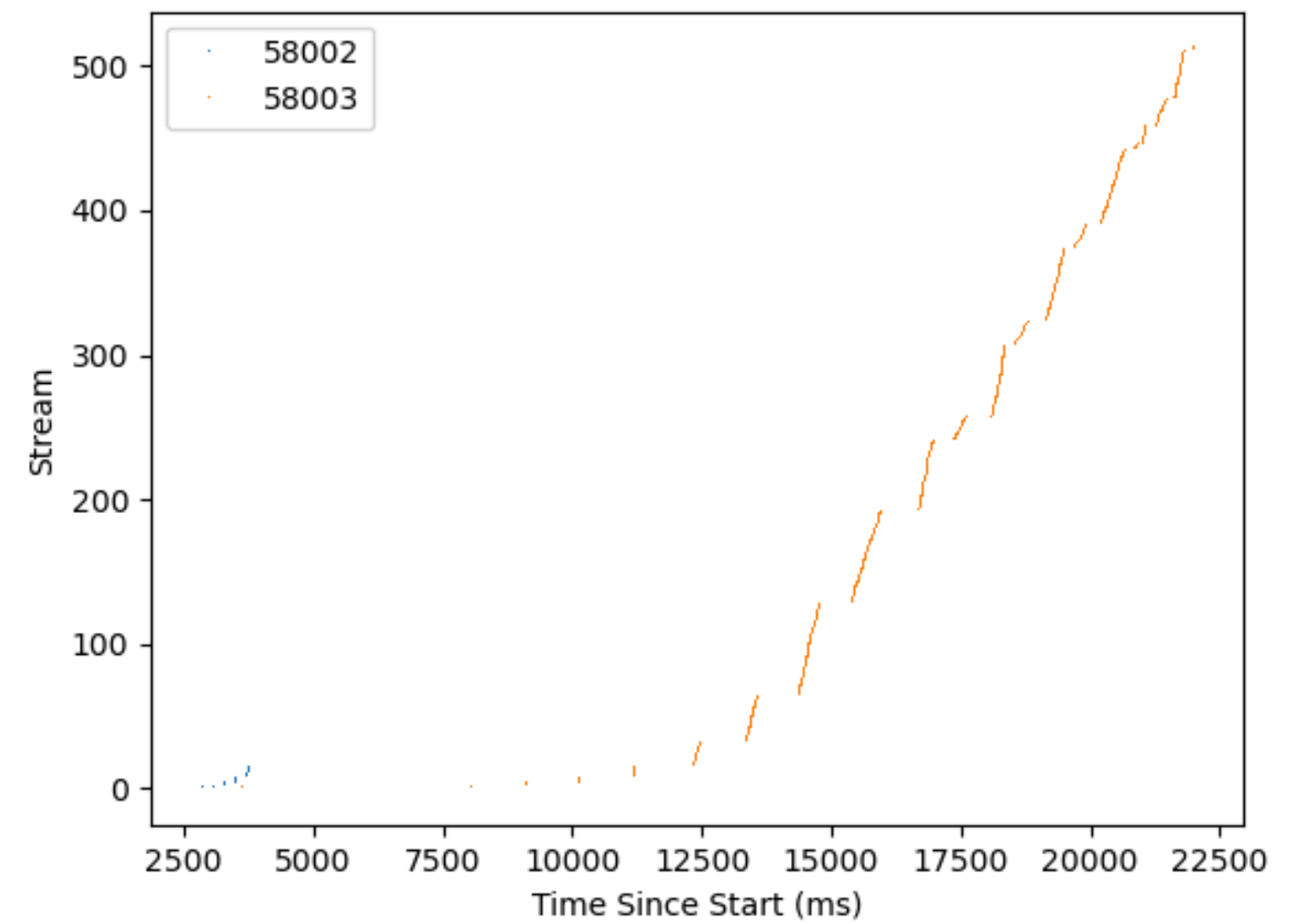
Concurrency check & Crash check

03

Concurrency check



Crash check



优化说明

Optimize check
从8min到2min

04

SR滑动窗口实现

缓存乱序包

```
elif seq_num > expected:
    # Out-of-order packet: buffer it
    buffer = self.state.out_of_order_buffer[session_key]
    is_new_packet = seq_num not in buffer

    if is_new_packet:
        buffer[seq_num] = payload
        self._log(
            f"DEBUG: 📦 Buffered out-of-order packet seq={seq_num}"
        )
    else:
        # Packet already in buffer (duplicate), no need to buffer
        self._log(f"DEBUG: 📄 Duplicate out-of-order packet seq={seq_num}")
```

交付乱序包

从缓存里交付的包不需要传递ack
在交付完所有缓存之后传递最后一个缓存包的ack

```
# Check buffer for subsequent packets
buffer = self.state.out_of_order_buffer[session_key]
while (expected + 1) in buffer:
    next_seq = expected + 1
    next_payload = buffer.pop(next_seq)

    self.state.receiving[chunk_hex] += next_payload
    session.expected_seq = next_seq + 1
    now = time.time()
    session.last_ack_time = now
    session.last_packet_time = now # Update packet time for buffered packets too

    expected = next_seq
    self._log(f"DEBUG: ✨ Delivered buffered packet seq={next_seq}, new expected={expected + 1}")

# Send ACK after processing in-order packet and all buffered packets
# This cumulative ACK tells sender the highest consecutive sequence number received
ack_pkt = _pack_header(PktType.ACK, HEADER_LEN, seq=0, ack=expected)
self.sock.sendto(ack_pkt, from_addr)
self._log(f"DEBUG: Sending ACK after buffered packets: {expected} to {from_addr}")
# Check if complete (after processing buffered packets)
current_expected = session.expected_seq
```


SR滑动窗口实现

seq=263丢包

缓存乱序包

接收到seq=263

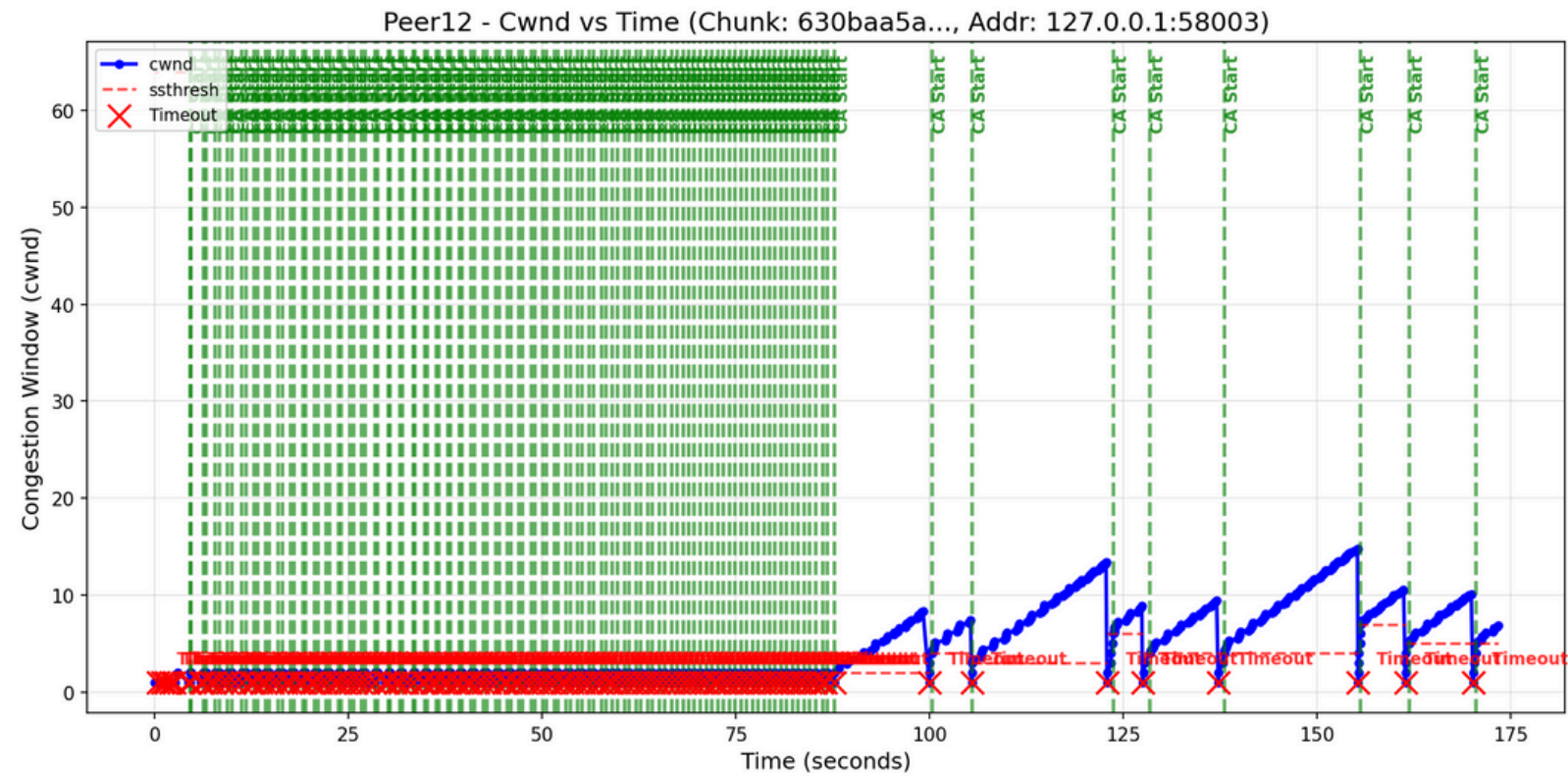
交付乱序包

传递交付后最新ack

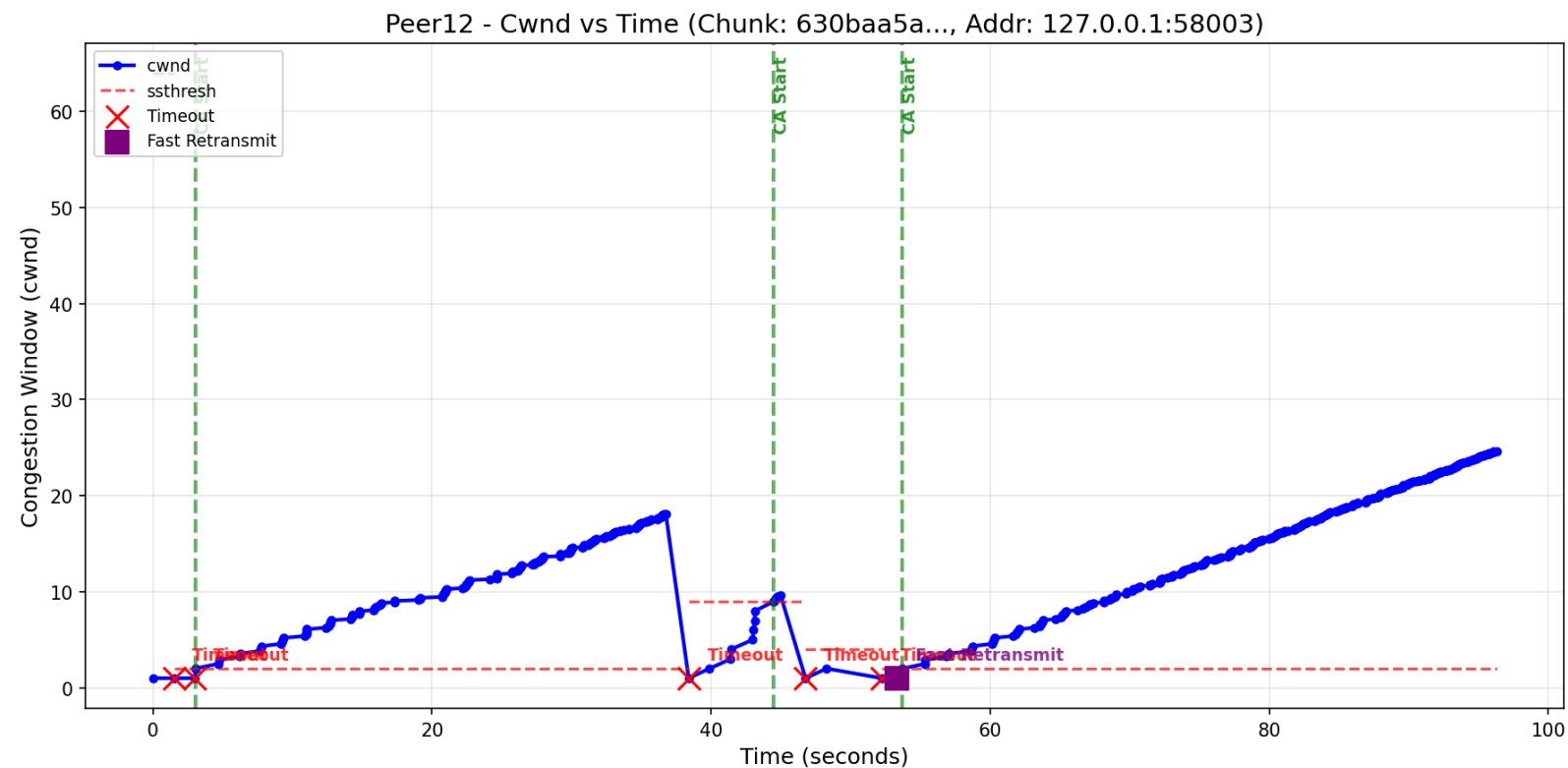
```
562 [Peer1@127.0.0.1:58001] DEBUG: Received seq=260, progress: 50.8% (266240/524288 bytes)
563 [Peer1@127.0.0.1:58001] DEBUG: Sending ACK after buffered packets: 260 to ('127.0.0.1', 58002)
564 [Peer1@127.0.0.1:58001] DEBUG: Received seq=261, progress: 51.0% (267264/524288 bytes)
565 [Peer1@127.0.0.1:58001] DEBUG: Sending ACK after buffered packets: 261 to ('127.0.0.1', 58002)
566 [Peer1@127.0.0.1:58001] DEBUG: Received seq=262, progress: 51.2% (268288/524288 bytes)
567 [Peer1@127.0.0.1:58001] DEBUG: Sending ACK after buffered packets: 262 to ('127.0.0.1', 58002)
568 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=264, expected=263, buffer_size=1
569 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=265, expected=263, buffer_size=2
570 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=266, expected=263, buffer_size=3
571 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=267, expected=263, buffer_size=4
572 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=268, expected=263, buffer_size=5
573 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=269, expected=263, buffer_size=6
574 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=270, expected=263, buffer_size=7
575 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=271, expected=263, buffer_size=8
576 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=272, expected=263, buffer_size=9
577 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=273, expected=263, buffer_size=10
578 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=274, expected=263, buffer_size=11
579 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=275, expected=263, buffer_size=12
580 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=276, expected=263, buffer_size=13
581 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=277, expected=263, buffer_size=14
582 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=278, expected=263, buffer_size=15
583 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=279, expected=263, buffer_size=16
584 [Peer1@127.0.0.1:58001] DEBUG: 📦 Buffered out-of-order packet seq=280, expected=263, buffer_size=17
585 [Peer1@127.0.0.1:58001] DEBUG: Received seq=263, progress: 51.4% (269312/524288 bytes)
586 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=264, new expected=265
587 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=265, new expected=266
588 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=266, new expected=267
589 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=267, new expected=268
590 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=268, new expected=269
591 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=269, new expected=270
592 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=270, new expected=271
593 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=271, new expected=272
594 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=272, new expected=273
595 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=273, new expected=274
596 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=274, new expected=275
597 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=275, new expected=276
598 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=276, new expected=277
599 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=277, new expected=278
600 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=278, new expected=279
601 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=279, new expected=280
602 [Peer1@127.0.0.1:58001] DEBUG: 🌟 Delivered buffered packet seq=280, new expected=281
603 [Peer1@127.0.0.1:58001] DEBUG: Sending ACK after buffered packets: 280 to ('127.0.0.1', 58002)
604 [Peer1@127.0.0.1:58001] DEBUG: Received seq=281, progress: 54.9% (287744/524288 bytes)
605 [Peer1@127.0.0.1:58001] DEBUG: Sending ACK after buffered packets: 281 to ('127.0.0.1', 58002)
606 [Peer1@127.0.0.1:58001] DEBUG: Received seq=282, progress: 55.1% (288768/524288 bytes)
```


超时检测灵敏度

修复前：在初始阶段大量重传



修复后：



在初始化时更宽容

```
session.estimatedRTT = est
session.devRTT = dev
# Use more conservative timeout: ensure minimum timeout and add safety margin
# For concurrent sessions, we need larger timeout to account for network competition
base_timeout = est + 4 * dev
# Ensure minimum timeout of 1.0s (instead of 0.5s) for better stability
# Add 20% safety margin for concurrent sessions
session.timeoutInterval = max(1.0, base_timeout * 1.2)
```

在高并发时更宽容

```
# For concurrent sessions, we need to be more conservative:
# - If we've sent packets beyond send_base, ACK might just be delayed
# - Use a more conservative timeout (1.5x) in this case
if next_seq_num > send_base:
    # We've sent packets beyond send_base, ACK might be delayed
    # Use more conservative timeout to avoid false positives
    effective_timeout = timeout * 1.5
else:
    # We haven't sent beyond send_base, use normal timeout
    effective_timeout = timeout
```

Thank you

恳请各位老师批评指正



第9组：廖子韬，胡耘宁，魏宇晴